

# Genie Campus Lab Match — PRD & TRD

Databricks Campus Hackathon — BMSCE Edition | Theme: Genie-Powered  
Campus Intelligence | Track A: Real-World Campus Problem Solver

---

## Part 1: Product Requirements Document (PRD)

### 1.1 Problem statement

Students seeking undergraduate research opportunities rely on two broken discovery channels:

- **Advisor referrals** — slow, and biased toward whoever the advisor personally knows, not toward actual fit.
- **Cold emails** — high rejection/no-response rate, because professors can't quickly assess fit from a generic message, and students often can't clearly articulate what a lab needs before applying.

The result: good student-lab matches go unmade simply because the discovery mechanism is broken, not because the interest or skill gap is real.

### 1.2 Product vision

A campus platform where students build a living profile through natural conversation (via Databricks Genie), browse labs in a marketplace-style interface with transparent fit and gap information, track applications, and return over time as their skills and the lab landscape evolve — turning lab discovery from network-gated to data-driven, without pretending to replace the human judgment that should decide final fit.

### 1.3 Target users

**Primary: Students** — undergrad/postgrad students (any year, any major with research interest) looking for a lab to join.

**Secondary: Faculty/PIs (or their lab managers)** — need a low-friction way to keep lab data current and review interested students.

### 1.4 Core user persona

**Ananya, 3rd-year CS undergrad.** Decent Python/DSA background, vaguely interested in "AI," no faculty connections, wants to join a lab before applying to grad school. Doesn't know how to articulate her interests precisely and doesn't want to be locked into a narrow lane too early.

## 1.5 Key user stories

1. As a student, I want to describe my skills and interests conversationally instead of filling out a form, so onboarding feels natural and low-effort.
2. As a student, I want to see *why* a lab is or isn't a good fit for me, not just a score, so I can make an informed decision.
3. As a student, I want to browse and filter labs like a marketplace, so I can explore beyond what the algorithm ranks first.
4. As a student, I want to see what skills I'm missing for a lab I'm interested in, so I know what to build toward.
5. As a student, I want to save labs and track my application status, so I have a reason to return to the app.
6. As a PI/lab manager, I want to update my lab's data conversationally, so keeping information current isn't a chore.
7. As a student, I want help drafting my application/outreach message (and answers to any extra questions a lab asks), so I don't have to write it from scratch — but I still want to review and edit it before it's sent.
8. As a PI/lab manager, I want to see how many students are viewing and matching with my lab, so keeping my data updated actually benefits me, not just the platform.

## 1.6 Feature set

### MVP (hackathon-critical):

- Conversational profile builder (Genie): skills, proficiency, interests, availability, academic year/major.
- Lab data store: research focus, required skills + depth, time commitment, faculty advisor, capacity/current team size, last-updated timestamp.
- Matching engine: skill fit, interest alignment, availability overlap → ranked results, shown with **explanations and gap breakdowns**, not just a score.

- Marketplace UI: browsable/filterable lab cards (area, commitment, "taking students"), each showing a **Ready now / Stretch pick** label + specific match/gap reasons + data freshness.
- **Apply Assist:** Genie drafts a personalized outreach message using the student's actual matched skills/interests (not generic text); if a lab has extra application questions, Genie asks the student conversationally and drafts answers from their real responses. Student always reviews and edits before anything is sent — deliberately **not** a fully autonomous auto-apply, to preserve authenticity toward faculty and keep a human accountable for what's sent (see 1.9 risks).

#### **High-priority stretch (build after MVP is stable, before other stretch items):**

- **PI mini-dashboard:** shows a lab's own view stats ("12 students viewed your lab this month, 4 are strong matches") — the incentive mechanism that makes keeping data updated worth a PI's time, rather than an unpaid chore.
- **Freshness + decay signal:** `last_updated` shown on every card; labs a student reports "applied, no response" to repeatedly get a quietly lowered reliability signal, self-correcting stale records using data you're already capturing.

#### **Other stretch (post-MVP / roadmap, mention in pitch even if not built):**

- Application status tracker (Applied → Pending → Interview → Decision).
- Saved-lab change alerts (new slot opened, data updated).
- Readiness-over-time tracking (gap shrinking as skills are added).
- Faculty-side Genie conversation for updating lab data.
- Lightweight PI-submitted "student quote" field for qualitative culture signal.

## **1.7 Non-goals**

- Not a replacement for the actual PI conversation/interview — the tool shortlists, humans decide.
- Not attempting to model lab "culture fit" algorithmically in v1.
- Not building a generic job-matching engine — explicitly tuned for the exploratory, interest-first nature of undergrad research (see intern feedback).

## **1.8 Success metrics (for pitch, even if not measurable in 12 hrs)**

- Time from "starts profile" to "sees ranked, explained shortlist" (target: under 3 minutes).
- % of matches where student proceeds to apply/contact (proxy for perceived fit quality).
- Data freshness distribution across labs (proxy for marketplace health).
- Return visits per student per semester (retention proxy).

## 1.9 Known risks

- **Cold-start / data staleness:** depends on faculty maintaining lab records. Mitigated by the PI mini-dashboard incentive (1.6) and the freshness/decay signal.
- **Matching quality unproven:** weights are heuristic, not calibrated against real outcomes yet.
- **No culture-fit signal:** mitigated by scope (shortlist tool, not decision-maker) and by an optional student-quote field.
- **Apply Assist authenticity risk:** a fully autonomous auto-apply could send generic, inauthentic messages that damage trust with faculty. Mitigated by design — draft-and-review only, never send without student approval.

### 1.9a Risk mitigation summary

| Risk                        | Mechanism                                                                                                                              | Hackathon reality                                                                                           |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Cold-start / data staleness | PI mini-dashboard (view stats as incentive) + visible <code>last_updated</code> + crowdsourced decay signal from "no response" reports | Seed 10–15 real-looking lab records for the demo; present the mechanism as the plan, not as solved adoption |
| Matching quality unproven   | Transparent per-component breakdown (skill/interest/availability) instead of one opaque score, so reasoning is checkable live          | Roadmap: calibrate against real application outcomes once available                                         |
| No culture-fit signal       | Explicit scope boundary — shortlist tool, not decision-maker; optional PI-submitted student quote for texture                          | Say this out loud rather than implying the tool judges fit fully                                            |
| Apply Assist authenticity   | Draft-and-review only — Genie never sends without student approval                                                                     | Guardrails + evals on the agent itself (2.6) back this                                                      |

| Risk | Mechanism | Hackathon reality                    |
|------|-----------|--------------------------------------|
|      |           | up technically, not just as a policy |

The consistent pattern: **name the limitation before a judge does, then show the specific mechanism** — this reads as maturity rather than a gap.

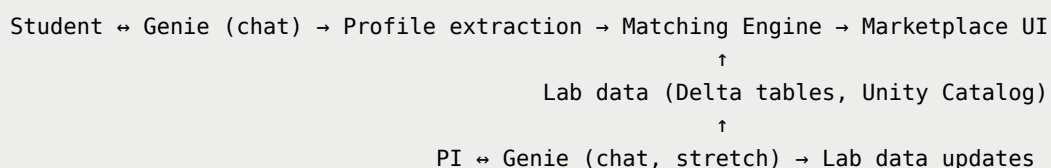
## 1.10 Differentiation ("wow factor")

With only 2 hackathon themes, many teams will land on a similar problem/solution shape. What should set this apart:

- **A validated pivot, stated explicitly.** Real research-intern feedback changed the design from a job-matching-style scorer to an exploration/gap-analysis tool. Most teams build on assumptions; this team corrected course based on real user input — say this out loud early in the pitch rather than leaving it implicit in the design.
- **Genie used natively, twice, visibly in the demo.** (1) Conversational profile intake, and (2) Apply Assist drafting a personalized outreach message — both real uses of Genie's conversational strength, not a chatbot wrapper around a generic backend.
- **Genie Spaces querying Delta tables directly** — lets a judge type their own natural-language question live (e.g. "which labs need Python and have an open slot") and get a real answer against real data, rather than only watching a scripted demo path.
- **Transparent gap reasoning over black-box scoring** — a visual skill-gap view (e.g. a small chart per lab, "you vs. what's required") is a concrete, memorable visual moment worth building if time allows, distinct from a text-only explanation.

## Part 2: Technical Requirements Document (TRD)

### 2.1 System overview



## 2.2 Databricks components (core to the build — lean into these for the pitch)

- **Databricks Genie (Genie Spaces)** — the conversational layer on *both* sides of the marketplace:
  - Student-facing: extracts skills, interests, availability from free-form chat; asks clarifying follow-ups on vague input (e.g. "AI" → probes model-building vs. data/language vs. systems).
  - PI-facing (stretch): same pattern, applied to keeping lab records current.
  - **Key design choice:** point Genie Spaces directly at your curated Delta tables (student profile table + lab table) so Genie itself can answer natural-language queries against structured data ("which labs need Python and have open slots?") — this is the most "Databricks-native" part of the build and worth foregrounding in the demo, since it uses Genie for what it's specifically built for rather than as a generic chatbot wrapper.
- **Delta Lake tables (Unity Catalog)** — the system of record for:
  - `student_profiles`: student\_id, skills (name, proficiency), interests (text + tags), availability\_hrs, academic\_year, major, last\_updated.
  - `labs`: lab\_id, PI, research\_focus, required\_skills (name, depth), time\_commitment, capacity, current\_team\_size, recent\_publications, last\_updated.
  - `applications` (stretch): student\_id, lab\_id, status, drafted\_message, timestamp, no\_response\_flag (feeds the decay signal in 1.6/1.9).
  - **Apply Assist flow:** reuses the same Genie Space as intake — a second conversation turn where Genie drafts an outreach message (and, if applicable, answers to lab-specific extra questions) using fields already in `student_profiles` and the target `labs` row. Output is written to `applications.drafted_message` for student review/edit before send — never sent automatically.
- **Databricks SQL warehouse** — serves the matching query layer (skill/interest/availability comparison) that powers the ranked marketplace results; keeps compute serverless and avoids standing up a separate DB for the hackathon.

- **(Optional stretch) Model Serving** — if time allows, host a small embedding model for semantic similarity between free-text student interests and lab focus descriptions, rather than keyword matching alone.

## 2.3 Application layer

- **Backend:** a thin API (FastAPI or similar) that:
  - Calls the Genie API to run the conversational extraction and structure the output into the profile schema.
  - Runs the matching query against Databricks SQL (skill overlap, interest similarity, availability overlap) and returns ranked + explained results.
  - Serves marketplace endpoints: list/filter labs, save lab, submit application (stretch).
- **Frontend:** marketplace grid (filter/sort/search), lab detail view (gap breakdown), chat panel (collapsible, used for onboarding/profile edits), profile summary bar.

## 2.4 Matching logic (transparent, not black-box)

Instead of a single opaque score, compute and surface three components per lab:

1. **Skill overlap:** student skills vs. required skills, per-skill match/gap, depth-aware.
2. **Interest alignment:** semantic similarity between stated interests and lab focus text (embedding similarity if time allows; keyword/tag overlap as fallback).
3. **Availability overlap:** student hrs/week vs. lab's stated commitment.

Output label logic (not a percentage):

- **Ready now:** skill overlap above threshold + availability fits + lab has open capacity.
- **Stretch pick:** meaningful interest alignment but a named skill gap — always paired with *what's missing*, never shown as a bare label.

## 2.5 Guardrails and evals

**Intake Genie (student & PI conversations) — stay-on-topic guardrails:**

- System-prompt scoping: the intake Genie Space is instructed to only discuss skills, interests, availability, academic background, and lab data — and to redirect (not answer) anything outside that scope (general Q&A, unrelated advice, attempts to change its role).
- Refuse to fabricate lab or student data not present in the Delta tables — if asked something the data can't answer, it should say so rather than guess.
- Basic input handling: ignore attempts to override its instructions via the chat itself (prompt-injection style messages), since students or PIs typing directly into the box is untrusted input.

**Apply Assist agent — additional guardrails (higher stakes, since output reaches a real professor):**

- **Grounding constraint:** every claim in a drafted message must trace back to a field actually present in `student_profiles` — no inventing projects, skills, or experience the student never stated.
- **Tone/content constraint:** draft stays factual and specific (referencing the real matched skills/interests), not generic filler or overstated enthusiasm.
- **Hard stop on autonomous send:** technically enforced, not just a UI choice — the send action requires an explicit student confirmation step; the agent has no path to email/submit without it.
- **Scope constraint:** only drafts outreach messages and answers to the target lab's actual listed questions — doesn't answer unrelated questions or take actions outside the application flow.

**Evals (even a lightweight version strengthens the pitch):**

- A small test set of student profiles × lab pairs, checking the intake Genie stays on-topic when given off-topic or adversarial prompts.
- A small test set checking Apply Assist drafts only reference facts present in the input profile (no fabrication) — even 5–10 manually reviewed examples is enough to show judges you tested this, not just built it.
- Track a simple pass/fail rate on these test sets; mention the numbers in the pitch as evidence of intentional safety design, not just a claimed guardrail.

## 2.6 Non-functional requirements

- Data freshness surfaced on every lab card (`last_updated` from the Delta table) — an honesty mechanism, not just a UI nicety.



- Matching query should run fast enough for a live demo (sub-2s) — keep it a single SQL query over indexed/small Delta tables for the hackathon scale, not a heavyweight ML pipeline.
  - No PII beyond what's needed for matching; student data isolated from being publicly queryable by other students.
- 

## Part 3: What to focus on for the 12-hour hackathon

### Build in priority order:

1. **Genie conversational intake → structured student profile** (this is your Databricks-differentiator; get it working end-to-end first, even with a small skill/interest vocabulary).
2. **Lab data in Delta tables + a Genie Space querying them** — this is the single most important thing to nail for a Databricks-sponsored hackathon, since it's the part that actually proves you used their product natively rather than bolting Genie onto a generic app.
3. **Matching query + transparent gap output** (skip the ML embedding stretch goal unless 1 and 2 are solid and stable — keyword/tag overlap is a fine fallback and still demoable).
4. **Marketplace UI**: even a simple filterable card grid is enough — this is what visually differentiates you from "just a chatbot," so don't cut it to save time on stretch features.
5. **One clean end-to-end demo path**: Ananya's persona flow, chat → profile → 3 ranked labs with gap reasoning → Apply Assist drafts a message → apply button. Rehearse this exact path; it's what the jury will see.
6. **Apply Assist (draft + review)** — build this next once the core loop is stable; it's your second live Genie moment and directly reflects real user feedback (research-intern interview), which strengthens the "validated pivot" narrative.
7. **If time remains**: let a judge type their own query live into the Genie Space against your Delta tables, and/or add a simple visual skill-gap chart per lab — these are your two strongest "wow factor" candidates (1.10) but are additive polish, not foundation.
8. **Guardrails on both Genie layers (2.5)** — even a basic system-prompt scope constraint on intake, and a grounding + no-autonomous-send

constraint on Apply Assist, should be built alongside those features, not skipped — this is a low-effort, high-credibility addition since AI safety is an easy, pointed question for a judge to ask.

**De-prioritize / cut if short on time:** application status tracker, notifications, readiness-over-time tracking, PI-side Genie flow, PI mini-dashboard, semantic embedding matching. Mention these as roadmap items in the pitch — judges respond well to "here's what's next" as long as the core loop works flawlessly live.

**In the pitch, explicitly say:**

- The validated pivot: real research-intern feedback moved you from job-style scoring to gap-based exploration — say this early, don't leave it implicit.
- Why this isn't job-matching (interest depth > pure skill overlap, exploratory not narrowing) — preempts the strongest critique before it's asked.
- Where Genie is doing real work (structured extraction, live Delta table queries, Apply Assist drafting), not just chat UI dressing.
- Why Apply Assist is draft-and-review, not autonomous — a deliberate trust decision, not a missing feature.
- The known limitation (data staleness) and your mitigation plan (PI incentive dashboard + freshness/decay signal) — shows maturity over pretending it's solved.
- The guardrails on both Genie layers (intake stays on-topic, Apply Assist is grounded and never sends autonomously) plus your small eval test set — even lightweight evals show intentional safety design if a judge asks "how do you know it behaves."